



Domain Specific Language Design (2IMP20)



Loek Cleophas



Ivan Kurtev



Felipe Xavier

About me



Dr.ir. Loek Cleophas
(l.g.w.a.cleophas@tue.nl)

Assistant professor,
Software Engineering
and Technology

+ Extraordinary
associate prof.

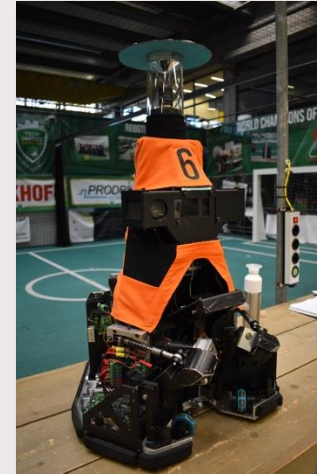


Research

- Model-Driven Engineering
- Engineering of Digital Twins

Teaching

- 2IMP20 DSL Design
- 2IRR80 Capstone Software Development



Goal of this course

The goal of this Master course is to *introduce the concepts needed when designing and developing domain specific languages.* (...)

The aim is that *at the end of the course you are able to design and implement a non-trivial domain specific language by yourselves.* This allows you to be better prepared for working in an environment that uses model-driven (software) techniques.

Learning objectives

1. understand and use *modern language workbenches* such as Rascal;
2. define *regular grammars*;
3. define *context-free grammars*;
4. understand formalisms to define *static and dynamic semantics* of a language;
5. define *abstract syntax definitions* in the form of metamodels;
6. define a *concrete textual syntax* for a language based on an abstract syntax definition;

Learning objectives

7. understand, develop and apply *model transformations*;
8. understand, develop and apply *code generators*;
9. develop *textual editors* for languages *within workbenches and tools such as Rascal and the Eclipse Modeling Framework*;
10. *design a small domain specific language* and create an *environment for editing and executing*.

Scheduling

- 2 x 2 hours lectures per week
 - ***Not streamed, not recorded***
- *Demo sessions in some lectures (Felipe)*
- *Walk-in Q&A sessions on Wednesdays lunch time*

Legend
No lecture
Loek
Ivan
Guest lecture
Backup slot

Wednesday (7th + 8th)	Friday (3rd + 4th)
April 22nd	April 24th
April 29th	May 1st
May 6th	May 8th
May 13th	May 15th
May 20th	May 22nd
May 27th	May 30th
June 3rd	June 5th
June 10th	June 12th
June 17th	June 19th

Course Outline

Grammarware part: basic generic language technologies — Loek

- Introduction to languages and DSLs
 - Rascal
- Lexical and context-free grammars by means of grammarware approach
- Semantics
- Design of domain specific languages

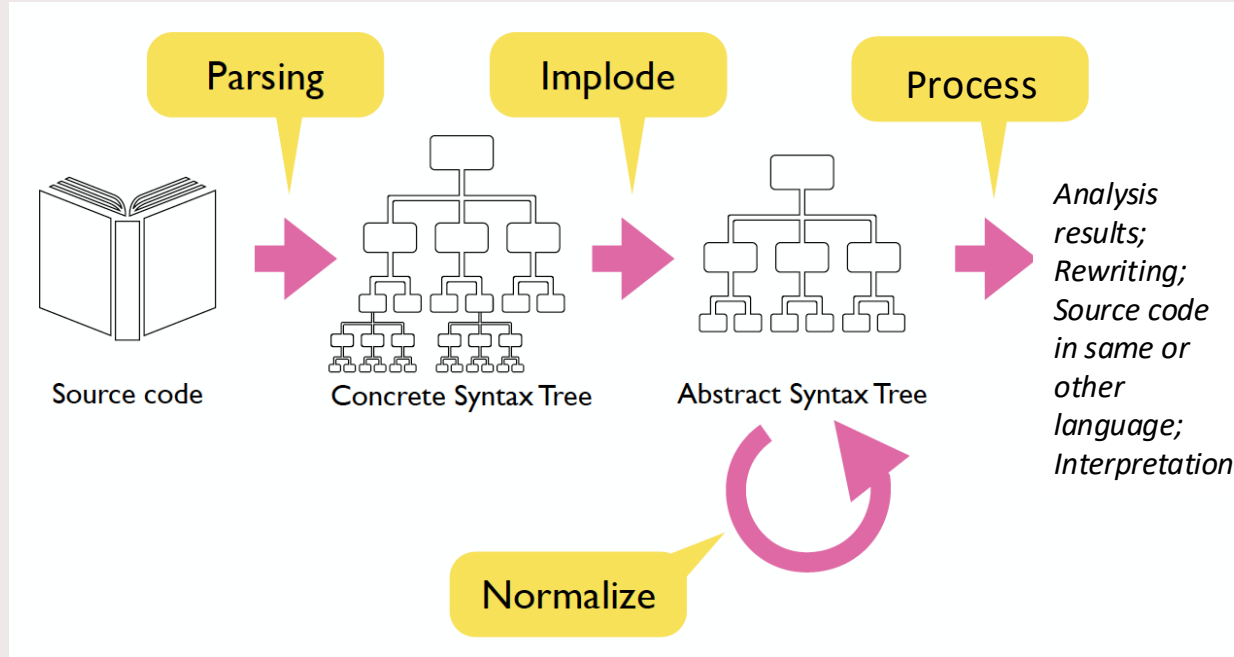
Modelware part: applied generic language technologies — Ivan

- Model-ware approach to define DSLs
 - Meta-modeling
 - XText for textual syntax
- Principle of model transformations
- Code generation

“The bigger picture”

- DSL Design
- State of the art research in DSL Design + Implementation
- MPS tool
- DSLs at Canon
- Possibly: DSLs & Twinning, Low code?

High-level: source code processing (both GPLs and DSLs)



Introduction to Rascal

Rascal =

Functional

Metaprogramming

Language

Introduction to Rascal

Rascal =

**Functional
Metaprogramming
Language**

- Immutable variables
- Higher order functions
- Static safety, with local type inference

Introduction to Rascal

Rascal =

Functional
Metaprogramming
Language



???

Introduction to Rascal



WIKIPEDIA
The Free Encyclopedia

Metaprogramming

From Wikipedia, the free encyclopedia

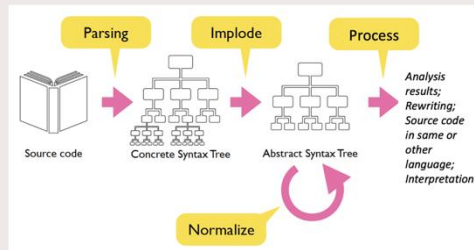
Metaprogramming is a programming technique in which computer programs have the [ability to treat other programs as their data](#). It means that a program can be designed to [read, generate, analyze or transform other programs](#) [...]

Introduction to Rascal

Rascal =
Functional
Metaprogramming
Language

- “Code as data”
- Program that generates/analyzes other programs
- Syntax definitions and parsing
- Pattern matching and rewriting mechanisms
- Visiting / traversal of Tree structure

High-level: source code processing (both GPLs and DSLs)



Introduction to Rascal

Rascal can be used for...

Forward Engineering
(Prototyping, DSL development)



This is our focus in this course



Backward Engineering
(Analysis, detectors, renovations)

Introduction to Rascal

1 Basic concepts

Introduction to Rascal

Functional immutability with an imperative syntax

- All data is immutable

Introduction to Rascal

Common Data Types

- e.g. Sets, Lists, Maps, Tuples and Relations
- can all be used in comprehensions

Introduction to Rascal

Sets

- **Unordered** sequence of values of the **same static type**
- All elements are **distinct**
- Allows all sorts of powerful operations like comprehensions, difference, slicing, etc..
- `import Set;` for convenient functions on sets
- See: <https://www.rascal-mpl.org/docs/Rascal/Expressions/Values/Set/>

```
rascal>{ N * N | int N <- [0 .. 10]};  
set[int]: {16,64,1,9,81,4,0,49,36,25}  
rascal>{ N * N | int N <- [0 .. 10], N % 3 == 0};  
set[int]: {9,81,0,36}
```

Introduction to Rascal

Lists

- Ordered sequence of values of the same static type
- Allows for duplicate entries
- Allows all sorts of powerful operations like comprehensions, difference, slicing, etc..
- `import List;` for convenient functions on lists
- See: <https://www.rascal-mpl.org/docs/Rascal/Expressions/Values/List/>

Introduction to Rascal

Source locations

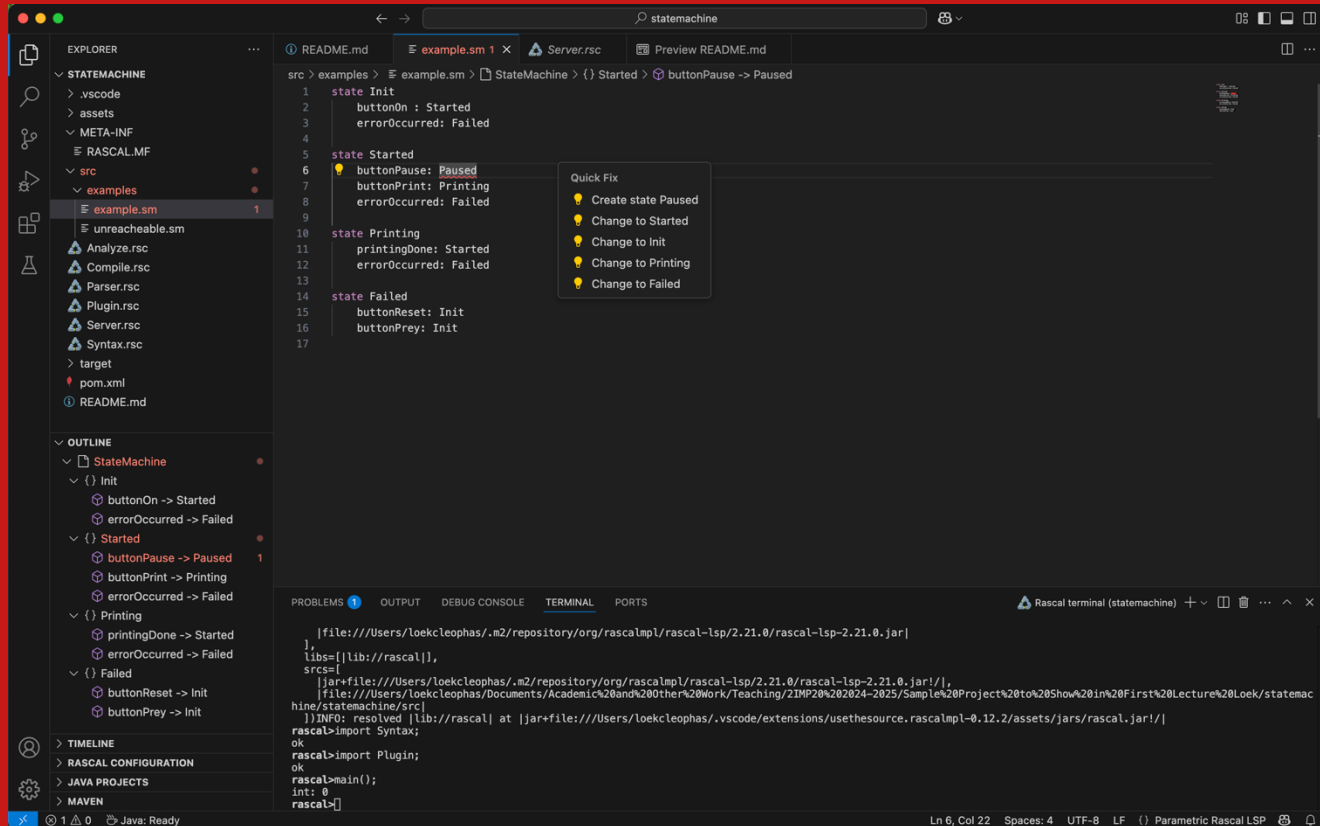
- Provide a **uniform way to represent files** on local or remote storage
- Can have different schemes
 - `file:!!!//`
 - `project:!//`
 - `http:!//`
 - etc ...
- Can contain **text location markers** (line + column, possibly for both begin and end)
- See: <https://www.rascal-mpl.org/docs/Rascal/Expressions/Values/Location/>

Introduction to Rascal

String templates

- Easy way to “generate” strings
- Often used for source-to-source transformation
- See: <https://www.rascal-mpl.org/docs/Recipes/Common/StringTemplate/>

A toy example in Rascal



Introduction to Rascal

Tuples

- Ordered sequence of elements
- Tuples are fixed sized
- Elements may be of different types
- Each element can have a label
- See: <https://www.rascal-mpl.org/docs/Rascal/Expressions/Values/Tuple/>

Introduction to Rascal

Relations

- Set of Tuples
- All elements have the same static tuple type
- Next to the set operations allows for composition, joining, transitive closure, etc
- `import Relation;` for convenient functions on relations
- See: <https://www.rascal-mpl.org/docs/Rascal/Expressions/Values/Relation/>

Introduction to Rascal

Pattern matching

- Determines **whether pattern matches a given value**
- One of Rascal's most powerful features
- Can bind matches to local variables
- Can be used in many places
- May result in multiple matches so employs local backtracking
- See <https://www.rascal-mpl.org/docs/Recipes/BasicProgramming/PatternMatching/>

Introduction to Rascal

Pattern

Match subject

Different types of matching

type-based matching

```
int x := 3;
```

structural matching

```
event(x, y) := event("a", "b");
```

anti-matching

```
event("c", "d") !:= event("a", "b");
```

list matching

```
[*x, 1, *y] := [5, 6, 1, 1, 1, 3, 4];
```

set matching

```
{1, *x} := {4, 5, 6, 1, 2, 3};
```

deep matching

```
/transition(e, "idle") := ast;  
/state(x, _, /transition(_, x)) := ast;
```

element matching

```
3 <- {1,2,3}  
int x <- {1,2,3}
```

regular expressions

```
/[A-Za-z]*/ := "09090aap noot mies"
```

Questions?

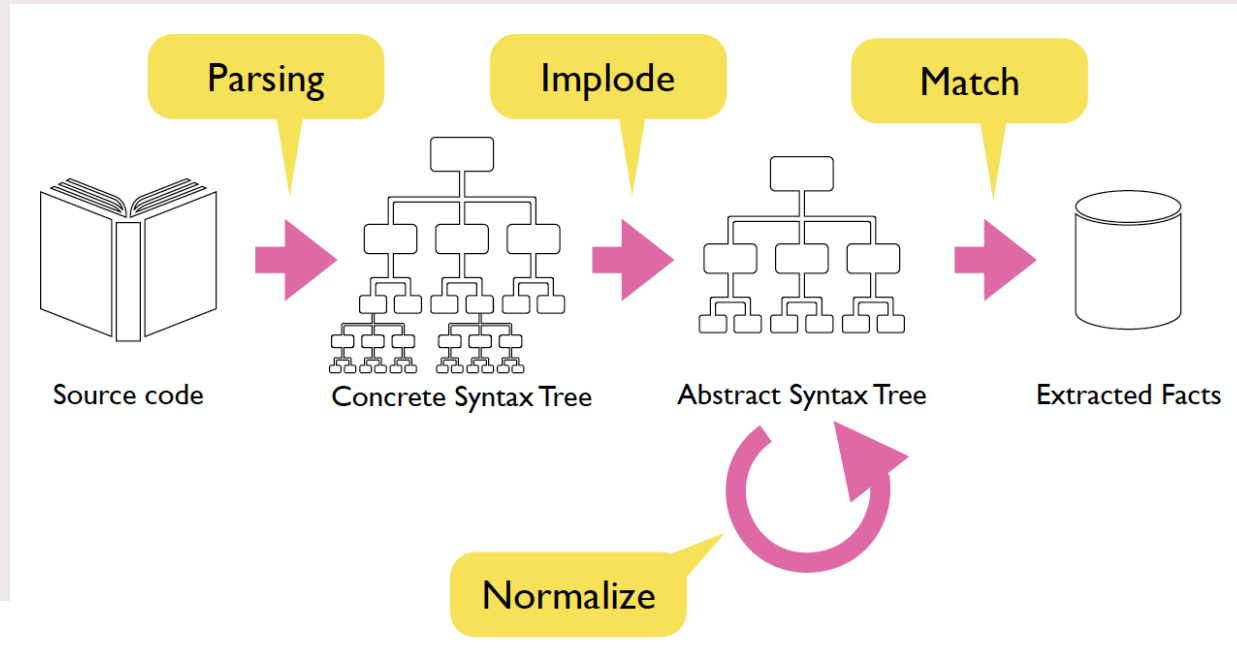


Introduction to Rascal

2 Tools for language engineering

Introduction to Rascal

Extracting facts using parsing



Introduction to Rascal

Concrete Syntax

```
start syntax Program = "begin" Stat* "end" ;
```

```
syntax Stat  
  = "if" Expression "then" Stat* "else" Stat* "fi"  
  | Id "!=" Expression  
  | "while" Expression "do" Stat* "od" ;
```

```
syntax Expression  
  = Id  
  | "(" Expression ")"  
  | left Expression "*" Expression  
  > left Expression "+" Expression ;
```

```
lexical Id = [A-Za-z][A-Za-z0-9\_-]*;
```

```
layout Whitespace = [\ \t\n\r]*;
```

Introduction to Rascal

Describes all possible strings which can be produced

Concrete Syntax

```
start syntax Program = "begin" Stat* "end" ;
```

```
syntax Stat  
  = "if" Expression "then" Stat* "else" Stat* "fi"  
  | Id ":@" Expression  
  | "while" Expression "do" Stat* "od" ;
```

```
syntax Expression  
  = Id  
  | "(" Expression ")"  
  | left Expression "*" Expression  
  > left Expression "+" Expression ;
```

```
lexical Id = [A-Za-z][A-Za-z0-9\_-]*;
```

```
layout Whitespace = [\ \t\n\r]*;
```

Introduction to Rascal

Describes all possible strings which can be produced

Concrete Syntax

```
start syntax Program = "begin" Stat* "end" ;
```

```
syntax Stat  
  = "if" Expression then Stat* else Stat* fi  
  | Id ":@" Expression  
  | "while" Expression "do" Stat* "od" ;
```

Nonterminal

```
syntax Expression  
  = Id  
  | "(" Expression ")"  
  | left Expression "*" Expression  
  > left Expression "+" Expression ;
```

```
lexical Id = [A-Za-z][A-Za-z0-9\_-]*;
```

```
layout Whitespace = [\ \t\n\r]*;
```

Introduction to Rascal

Describes all possible strings which can be produced

Concrete Syntax

```
start syntax Program = "begin" Stat* "end" ;
```

Terminal

```
syntax Stat  
  = "if" Expression "then" Stat* "else" Stat* "fi"  
  | Id ":@" Expression  
  | "while" Expression "do" Stat* "od" ;
```

Nonterminal

```
syntax Expression  
  = Id  
  | "(" Expression ")"  
  | left Expression "*" Expression  
  > left Expression "+" Expression ;
```

Terminal

```
lexical Id = [A-Za-z][A-Za-z0-9\-\]*;
```

```
layout Whitespace = [\ \t\n\r]*;
```

Layout characters

Introduction to Rascal

Abstract Syntax (= ADT)

```
data Program = program(list[Stat] stats);

data Stat =
  \if(Expr cond, list[Stat] \tr, list[Stat] \f)
  | assign(str id, Expr val)
  | \while(Expr cons, list[Stat] body) ;

data Expr =
  id(str name)
  | mult(Expr lhs, Expr rhs)
  | add(Expr lhs, Expr rhs) ;
```

Introduction to Rascal

Same information as concrete tree but more abstract (ADT)

```
data Program = program(list[Stat] stats);

data Stat =
  \if(Expr cond, list[Stat] \tr, list[Stat] \f)
  | assign(str id, Expr val)
  | \while(Expr cons, list[Stat] body) ;

data Expr =
  id(str name)
  | mult(Expr lhs, Expr rhs)
  | add(Expr lhs, Expr rhs) ;
```

Introduction to Rascal

Same information as concrete tree but more abstract (ADT)

```
data Program = program(list[Stat] stats);
```

data Abstract Data Type

```
\if(Expr cond, list[Stat] \tr, list[Stat] \f)  
| assign(str id, Expr val)  
| \while(Expr cons, list[Stat] body) ;
```

```
data Expr =  
  id(str name)  
  | mult(Expr lhs, Expr rhs)  
  | add(Expr lhs, Expr rhs) ;
```

Introduction to Rascal

Same information as concrete tree but more abstract (ADT)

```
data Program = program(list[Stat] stats);
```

data Abstract Data Type

```
\if(Expr cond, list[Stat] \tr, list[Stat] \f)  
| assign(str id, Expr val)  
| \while(Expr cons, list[Stat] body) ;
```

data Escape for keywords

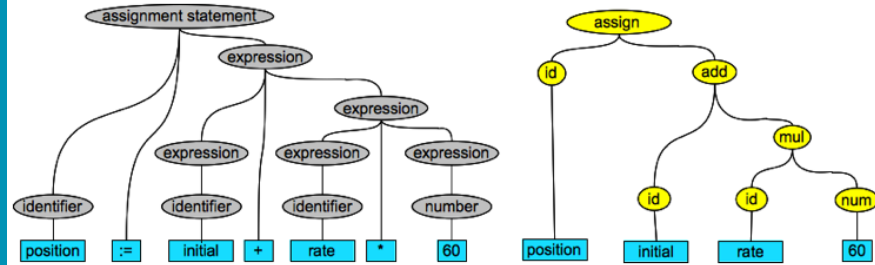
```
id(str name)  
| mult(Expr lhs, Expr rhs)  
| add(Expr lhs, Expr rhs) ;
```

Concrete vs. Abstract

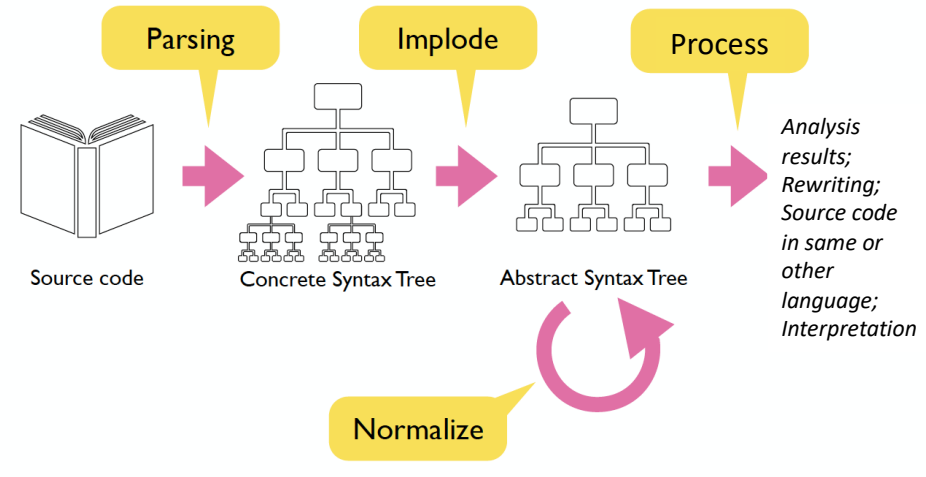
For the input sentence

position := initial + rate * 60

the parse tree (left) and abstract syntax tree (right) may look as follows:



(Source: <https://www.rascal-mpl.org/docs/Rascalopedia/ParseTree/>)



Questions?



Introduction to Rascal

Extracting facts from an Abstract Syntax Tree

- Use Pattern Matching
 - Match on structure
 - Match on values
 - Deep matching
- See: <https://www.rascal-mpl.org/docs/Recipes/BasicProgramming/PatternMatching/> and <https://www.rascal-mpl.org/docs/Rascal/Patterns/>

Introduction to Rascal

Find all assigned variables

```
data Program = program(list[Stat] stats);

data Stat =
  \if(Expr cond, list[Stat] \tr, list[Stat] \f)
  | assign(str id, Expr val)
  | \while(Expr cons, list[Stat] body) ;

// find assigned identifiers
for (Stat s <- program.stats, assign(str id, Expr _) := s) {
  println("Found id: <id> ");
}
```

Introduction to Rascal

Find all assigned variables

```
data Program = program(list[Stat] stats);
```

```
data Stat =  
  \if(Expr cond, list[Stat] \tr, list[Stat] \f)  
  | assign(str id, Expr val)  
  | \while(Expr cons, list[Stat] body) ;
```

Iterate over all Stats in subtree

Only match on 'assign'

```
// find assigned identifiers  
for (Stat s <- program.stats, assign(str id, Expr _) := s) {  
  println("Found id: <id> ");  
}
```

Wildcard

Introduction to Rascal

Find all assigned variables

```
data Program = program(list[Stat] stats);

data Stat =
  \if(Expr cond, list[Stat] \tr, list[Stat] \f)
  | assign(str id, Expr val)
  | \while(Expr cons, list[Stat] body) ;
```

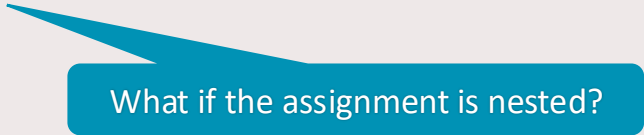
Direct iterate and match on `assign`

```
// find assigned identifiers
for (assign(str id, Expr _) <- program.stats) {
  println("Found id: <id> ");
}
```

Introduction to Rascal

Find all assigned variables

```
data Program = program(list[Stat] stats);  
  
data Stat =  
  \if(Expr cond, list[Stat] \tr, list[Stat] \f)  
  | assign(str id, Expr val)  
  | \while(Expr cons, list[Stat] body) ;
```



What if the assignment is nested?

Introduction to Rascal

Find all assigned variables (including *nested* ones)

```
data Program = program(list[Stat] stats);

data Stat =
  \if(Expr cond, list[Stat] \tr, list[Stat] \f)
  | assign(str id, Expr val)
  | \while(Expr cons, list[Stat] body) ;
```

Use a deep match

```
for (/assign(str id, Expr _) <- program) {
  println("Found id: <id>");
}
```

Introduction to Rascal

Find the expression assigned to the variable named "x"

```
data Program = program(list[Stat] stats);

data Stat =
  \if(Expr cond, list[Stat] \tr, list[Stat] \f)
  | assign(str id, Expr val)
  | \while(Expr cons, list[Stat] body) ;
```

Match on a specific value

```
for (/assign(("x"), Expr expr) <- program) {
  println("Expr assigned to x: <expr>");
}
```

Introduction to Rascal

Transforming an Abstract Syntax Tree

- Use **visit** statement
 - Reach all nodes
 - Not composable
- See <https://www.rascal-mpl.org/docs/Rascal/Expressions/Visit/>

Introduction to Rascal

Rename "x" to "y"

```
data Program = program(list[Stat] stats);

data Stat =
  \if(Expr cond, list[Stat] \tr, list[Stat] \f)
  | assign(str id, Expr val)
  | \while(Expr cons, list[Stat] body) ;

data Expr =
  = id(str name)

p = visit(p) {
  case assign("x", Expr val) => assign("y",val)
  case id("x") => id("y")
}
```

Introduction to Rascal

Rename "x" to "y"

```
data Program = program(list[Stat] stats);

data Stat =
  \if(Expr cond, list[Stat] \tr, list[Stat] \f)
  | assign(str id, Expr val)
  | \while(Expr cons, list[Stat] body) ;

data Stat = visit all the nodes in the tree
  = id(str id, Expr val)

p = visit(p) {
  case assign("x", Expr val) => assign("y", val)
  case id("x") => id("y")
}
```

Rewrite/replace node

Both assignment and use are replaced

Introduction to Rascal

Visiting strategies

- **top-down**: root to leaves (default)
- **top-down-break**: root to leaves but stop on case match
- **bottom-up**: leaves to root
- **bottom-up-break**: leaves to root but stop on case match
- **innermost**: bottom-up fix point (repeat until no more changes)
- **outermost**: top-down fix point

Questions?



Introduction to Rascal

Debugging Rascal:

- Poor-mans debugging
 - Using `println` or `bprintln` (in comprehensions)
 - You need to import IO for this!
- Rich-mans debugging
 - Using the debugger
 - You have to open the debug perspective manually!

Introduction to Rascal

Common errors: Undeclared variable

- Forgetting to import a module, i.e.:
 - Function is declared private

```
rascal> l = [1,2,3];  
list[int]: [1,2,3]  
rascal> size(l);  
|prompt:///| (0,4,<1,0>,<1,4>): Undeclared variable: size  
Advice: |http://tutor.rascal-impl.org/Errors/Static/UndeclaredVariable.html|
```

- Solution for above example: `import List;`

Introduction to Rascal

Common errors: **CallFailed**

- Calling a function with the wrong arguments

```
void someFunc(str a) {  
    println(a);  
}  
rascal> someFunc("a");  
a  
ok  
rascal> someFunc(2);  
|prompt:///| (9,1,<1,9>,<1,10>): CallFailed(  
|prompt:///| (9,1,<1,9>,<1,10>),  
[2])  
at $root$(|prompt:///| (0,12,<1,0>,<1,102))
```

Introduction to Rascal

Common errors: Root cause analysis

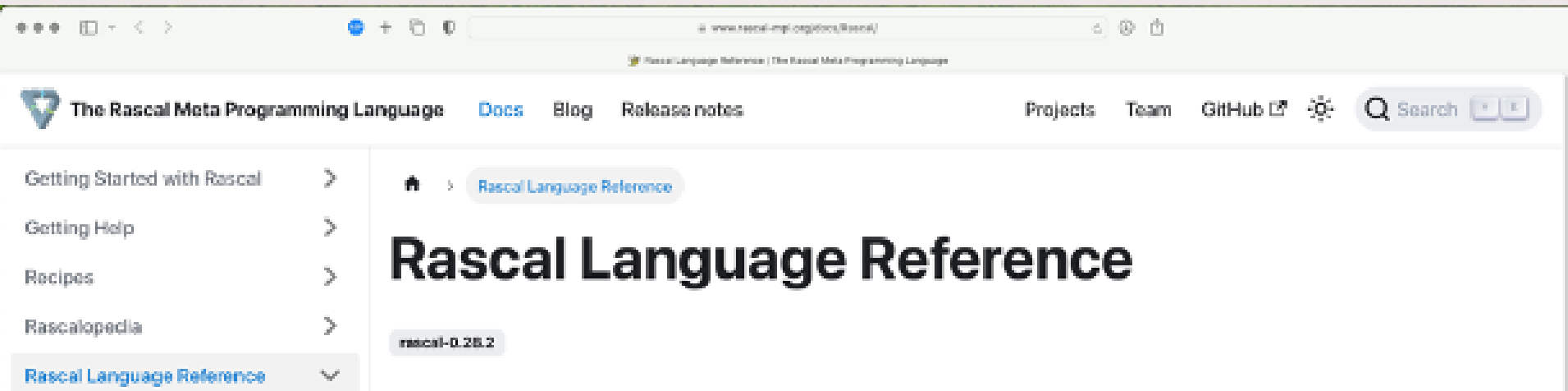
- Slice your problem by
 - Importing the problematic module directly
 - Use 'delta'-debugging (comment out 50% of the code and try to reimport, add 25% again and try again, etc)

Introduction to Rascal

Looking for how you can do stuff in Rascal?

Browse and search the documentation

- <https://www.rascal-mpl.org/docs/Rascal/>
- <https://www.rascal-mpl.org/docs/GettingStarted/>



The screenshot shows a web browser displaying the Rascal Language Reference page. The browser's address bar shows the URL <https://www.rascal-mpl.org/docs/Rascal/>. The website's header includes the Rascal logo, the text "The Rascal Meta Programming Language", and navigation links for "Docs", "Blog", and "Release notes". On the right side of the header, there are links for "Projects", "Team", "GitHub", and a search bar. The left sidebar contains a list of navigation items: "Getting Started with Rascal", "Getting Help", "Recipes", "Rascalopedia", and "Rascal Language Reference" (which is highlighted). The main content area features a breadcrumb trail "Home > Rascal Language Reference" and a large heading "Rascal Language Reference". Below the heading, the version "rascal-0.28.2" is displayed.

Introduction to Rascal

Looking for how you can do stuff in Rascal?

Take a look at the Rascal Cheatsheet:

Rascal Cheat Sheet
<http://www.rascal-mpl.org>
<http://tutor.rascal-mpl.org>
<https://github.com/cwi-swat/rascal>



Modules
module Example
import ParseTree; // import
extend lang::std::Layout; // "inherit"

Declarations
// Algebraic data types (ADT)
data Exp
= var(str x) // unary constructor
| add(Exp l, Exp r); // binary constructor
data Person // keyword parameter
= person(int id, bool married=false);
alias Age = int; // type alias
anno loc Exp@location; // annotation
private real PI = 3.14; // variables
// Functions: signatures are lists of patterns
// May have keyword parameters.
void f(int x) { println(x); } // block style
int inc(int x) = x+1; // rewrite style
int inc@int(x) = x+1 when x == 0; // side condition
default int inc@int(x) = x; // otherwise
// Test functions (invoke from console with :test)
test bool add() = 1+2 == 3;
// randomized test function
test bool com(int x, int y) = x*y == y*x;
// Foreign function interface to Java
@javaClass(name.of.javaClass.with.Method)
java int method();

Content-free grammars
start syntax Prog // start symbol
= prog: Exp exps // production
| stats: (Stat ";")* // separated list
| stats: (Stat ";")* // one-or-more sep. list
| "private"? Func; // optional
syntax Exp
= var: Id
| left mul: Exp l "*" Exp r // or right, assoc
| left div: Exp l "/" Exp r // reject
> left add: Exp l "+" Exp r // ">" = priority
| bracket "(" Exp ")";
lexical Comment
= "*" "[\n]* \$"; // begin/end markers
lexical Id
= "[a-zA-Z][a-zA-Z0-9_]*" // look behind restriction
[a-zA-Z][a-zA-Z0-9_]* // character classes
!> [a-zA-Z0-9_]* // lookahead restriction
Reserved; // subtract keywords
Layout Layout // for whitespace/comments
= "\ \t\n\r";
keyword Reserved // keyword class
= "if" | "else"; // finite langs
Statements
// Standard control-flow
if (E) S; else S;
while (E) S;
do S; while(E);
continue; break;
return; return E;
// Loop over all bindings produce by patterns
for (l < [0..10]) S; // Loop 10 times
fail; // control backtracking
append E; // add to loop result list

Pattern-based Switch-case
switch (E) {
case P: S; // do something
case P => E // rewrite it
default: S; // otherwise
}
// Traversal with visit; like switch, but matches
// at arbitrary depth of value
visit (E) {
case P: S; // do something
case P => E // rewrite something
case P => E when E
} insert E; // rewrite subject as statement
// Strategies: bottom-up, innermost, outermost,
// top-down-break, bottom-up-break
top-down visit (E) {}
try S; // pattern-based try-catch
catch P: S; // match to catch
finally S;
throw E; // throw values
// Fix-point equation solving;
// iterates until all params are stable
solve (out,ins) {
out[b] = { () | it = ins[s] | s <- succ[b] };
ins[b] = (out[b] - kill[b]) + gen[b];
};
x = 1; // assignment
nums[0] = 1; // subscript assignment
nums[1,3..10] = 2; // sliced (see below)
p.age = 31; // field assignment
ast@location = l; // annotation update
->p, @ = "<rd", 30; // destructuring
// A op=E == A = A op E
A += E; A -= E; A *= E;
A /= E; A ^= E;

<https://github.com/cwi-swat/rascal-cheat-sheet/raw/master/sheet.pdf>

Introduction to Rascal

Looking for how you can do stuff in Rascal?

Search on StackOverflow

<https://stackoverflow.com/questions/tagged/rascal>

https://stackoverflow.com/questions/tagged/rascal

Bookmarks Save to Mendeley Using Decision Ru... Test voor internet... Advice to new Pro...

stackoverflow [rascal]

Make your voice heard. [Take the 2019 Developer Survey now](#)

Home

PUBLIC

Stack Overflow

Tags

Users

Jobs

Teams
Q&A for work

Questions tagged [rascal]

Rascal is an experimental domain specific language for metaprogramming, such as static code analysis, program transformation and implementation of domain specific languages. It includes primitives from relational calculus and term rewriting. Its syntax and semantics are based on procedural (imperative) and functional programming.

[Watch Tag](#) [Ignore Tag](#) [Learn more...](#) [Improve tag info](#) [Top users](#) [Synonyms](#)

314 questions

info **Newest** Frequent Votes Active unanswered

1 [how to use "IO actions" with list comprehension or mapper in Rascal?](#)

BLOG

[How the 2019 Stack Overflow Developer Survey Came to Be Your Last Chance...](#)

FEATURED ON META

[Take the 2019 Developer Survey now](#)

HOT META POSTS

17 [When logged in, pages from are now synonyms do not re](#)

14 [Please don't auto-subscribe alerts](#)

Introduction to Rascal

Some “hello world” exercises on Rascal:

<https://www.rascal-mpl.org/docs/Recipes/BasicProgramming/>

Next lectures by me: Concrete + Abstract Syntax

